

Demo 2 feedback — approach plan

How we respond to every requirement and concern raised in the second demo (29 September 2026): what already exists in the code, the approach for each, dependencies and sequencing.

Summary

The report lists 24 items and 5 concerns. We checked each against the current code. They group into **ten workstreams**. Three things shape the plan:

● **The client's Tier 1 asks depend on their Tier 2 foundations.** XML returns, snapshot reporting, module FTE and the Engagement entity all need the effective-dated, versioned, multi-entity model first. We build the foundations first, but in thin vertical slices so something from Tier 1 is visible at each demo.

● **Some asks are partly built already.** Programme-change history already splits the return into periods (when generated for an academic year); past returns are stored; milestones, cohort import and the advisory feature are done; the integration hub, outbox, webhooks and a feature-flag precedent exist.

● **One conflict to reconcile, not a rework.** The client agreed one install per customer; we built shared-database multi-tenancy with row-level security. A per-customer install is simply one tenant per VM — no code change — and we keep the option of shared hosting.

Workstreams

WS	Workstream	Report items	Tier	Size
1	Curriculum catalogue & versioning	1.1, 1.3, terminology	2	L
2	Module & course statutory data, credits, FTE	1.2, 1.5, 1.6	1	M
3	Effective dating everywhere	1.4, 1.13, 4.3, 4.5	1–2	L
4	HESA entity coverage & snapshot reporting	1.7, 1.12, 4.4	1	L
5	Return output, amendments & submission	1.10, 1.11, 1.20	1 / 3	M
6	Integration API & data publication	1.9	1	L
7	Custom attribute governance	1.18, 4.1	2	M
8	Schema, HESA coverage & architecture documentation	1.8, 4.1, 4.2	2	S
9	Platform: installs, feature flags, hosting, regulatory updates	1.21–1.23	4	M
10	Carry-overs & next demo	1.14–1.17, 1.19, 1.24	3	S

Sizes are relative (S < M < L), for sequencing, not estimates.

Contents

1. Coverage matrix — every report item against the current code
2. Status in detail — implemented, partly implemented, not implemented (with approach)
3. Workstreams WS1–WS10 — current state, approach, deliverables
4. Dependencies and sequencing
5. Phased roadmap
6. Questions to confirm with the client
7. Risks
8. Sources

1. Coverage matrix

Status is what the code does today, checked on 30 Sep 2026. Green = built, amber = partly built, red = missing.
Totals: **6 implemented, 10 partly implemented, 8 not implemented.**

#	Item	Current state in code	WS
1.1	Module & programme catalogue	Modules always created inside a programme (programme_id required); electives can be shared via ModuleOffering	WS1
1.2	Statutory data at module & course level	No HESA codes on programme or module; spec marks COURSETYP / STUDYLEVEL “not yet captured”	WS2
1.3	Module versioning with effective dates	No version, run or effective dates on modules or programmes; only free-text term	WS1
1.4	Effective dating across the model	Lifecycle events, supervisors, funding dated; status, modules, UOA not	WS3
1.5	FTE at module level + validation	Student FTE only (lifecycle events); no module FTE; no check	WS2
1.6	Credits at module level	TaughtModule.credits exists; programme total credits exist	WS2
1.7	HESA Engagement entity	Only a flat student record; engagement events exist (meetings, milestones) but no attendance / VLE	WS4
1.8	Schema-to-HESA mapping document	Record-schema catalog + spec hints exist in code; no document	WS8
1.9	Warehouse export + object API	Integration hub, outbox, webhooks, adapters exist; no external API keys, no scheduled publication	WS6
1.10	XML output (+ CSV/Excel)	CSV only; no XML, no Excel	WS5
1.11	Edit before submission	Past returns stored and re-downloadable; not editable; no amendment flow	WS5
1.12	Snapshot as at a date	Return reads current values; no as-at date	WS4
1.13	Programme change in the return	Return already splits by programme period when an academic year is supplied — verify why the demo didn't show it	WS3
1.14	Cohort import refinements	CSV + grid, defaults, configurable template, duplicate detection	WS10
1.15	Single student enrolment	Taught and research; some fields come from other systems	WS10
1.16	Lifecycle changes	Intensity, mode, programme transfer with approval and impact preview	WS10
1.17	HESA advisory feature	Upload / paste ingest, mapping, cleansing, defaults	WS10
1.18	Custom attributes	Built; no approval gate, no usage tracking; delete doesn't check mappings	WS7
1.19	Milestones	Built (templates per programme, per-student panel) — not demoed	WS10
1.20	Direct API submission to HESA	Not built; HESA API availability to confirm	WS5
1.21	Regulatory updates under subscription	Spec packs are global (not tenant) tables — fits central release	WS9

#	Item	Current state in code	WS
1.22	Federated installs + features off by default	Shared-DB multi-tenancy built; composer .enabled is the only off-by-default feature	WS9
1.23	Oracle Cloud hosting	No OCI deployment material	WS9
1.24	Next demo: finance payments flow	Finance adapter exists in the integration hub	WS10

Concerns (section 4 of the report)

Concern	Addressed in
4.1 The “1000 custom attributes” problem	WS7 (governance) + WS8 (architecture document)
4.2 AI-generated code quality	WS8 (architecture + QA evidence)
4.3 Effective-dating coverage	WS3
4.4 Engagement and wider HESA entities	WS4
4.5 Programme change not in the statutory layer	WS3 (quick verification first)

2. Status in detail

The same 24 items, split by status. For partly and not implemented items, the approach column summarises the design; the workstream sections that follow give the detail.

2.1 Implemented (6)

#	Item	What exists today
1.6	Credits at module level	TaughtModule.credits, plus a credit total on the programme; module credits are checked against the programme total when the award is calculated
1.14	Cohort import	CSV upload and in-app grid, defaults for blank cells, configurable template (Settings → Cohort Import), duplicate detection
1.15	Single student enrolment	Direct enrolment into taught or research programmes; core modules auto-enrolled for taught students
1.16	Lifecycle changes	Intensity, study mode and programme transfer (including taught ↔ research), with requester → approver workflow and before/after impact preview
1.17	HESA advisory feature	URL / file / pasted-text ingest, mapping view, auto-map suggestions, cleansing, defaults, allowed values, required flags
1.19	Milestones	Templates per programme, auto-generated per student, student milestones panel. Built but not shown in the demo — it just needs demoing

2.2 Partly implemented (10)

#	Item	What exists	What's missing	Approach
1.1	Module & programme catalogue	Modules have a “home” programme; electives shared via <code>ModuleOffering</code>	Modules can't exist on their own; always created inside a programme	Modules become first-class catalogue entries; a programme ↔ module link (core / optional) replaces the home link and offerings; existing data migrated
1.4	Effective dating everywhere	Dated: lifecycle events, supervisors, funding, placements	Student status, module enrolments, UOA; the generic student edit overwrites status / programme with no history	Standard history tables (<code>valid_from</code> / <code>valid_to</code> / <code>recorded_at</code>), one write path, cached current value. Full design: Effective_Dating_Approach.pdf
1.8	Schema-to-HESA mapping document	Record-schema catalog and spec-field hints in code; mapping screen	No document for the client	Generated from the spec pack and mappings: each HESA field → internal source → covered / not covered; regenerated so it stays current
1.9	Warehouse export + object API	Integration hub, adapters, outbox, signed webhooks, OpenAPI docs	No API keys / service accounts for third parties; no scheduled publication	(a) Versioned external API per object with service accounts, scopes, rate limits, audit; writes go through normal services. (b) Admin-configured publications (object, fields, frequency, destination), incremental, run by the worker from a read-only session
1.11	Edit before submission	Every generated return stored and re-downloadable	Not editable; no amendment flow; validation report and as-at date not stored with it	Each return an immutable version. Amendments override specific fields on a previous version (with a reason), rebuilt from that snapshot so newer changes don't creep in. Raw-XML re-upload as an audited escape hatch
1.13	Programme change in the return	Return splits into programme periods when generated for an academic year	Didn't show in the demo	Quick win: check whether the demo return was generated without a year or the profile lacks per-period fields; fix. Then move to history tables
1.18	Custom attributes	Create with mandatory reason; value grid; mapped as <code>custom.<key></code>	No approval, no usage tracking; delete doesn't check mappings	Now: delete guard. Then the approval gate: duplicate / derivable check, maker–checker, “where used” usage map, core vs organisation-specific, load only used attributes
1.21	Regulatory updates under subscription	Spec packs are global tables, not per-tenant	No release process	Ship each HESA cycle's spec pack centrally, versioned; institutions adopt through the existing advisory / spec-version flow
1.22	Federated installs + features off by default	Shared-database multi-tenancy with RLS; <code>composer.enabled</code> is one off-by-default flag	No general feature-flag mechanism	One tenant per customer VM (no code change; RLS kept). A feature registry: each new capability has a flag, default off, admin-console toggle, enforced in API and UI

#	Item	What exists	What's missing	Approach
1.24	Next demo: finance payments	Finance adapter in the integration hub	No payments flow; no GrantsNow connection	Student → pre-award / post-award (GrantsNow) → payments through the integration layer; needs GrantsNow endpoints and a test environment

2.3 Not implemented (8)

#	Item	Approach	Depends on
1.2	Statutory data at module & course level	Course and module HESA fields defined by the spec pack, not hard-coded. Common ones (level, mode, credits, FTE) as columns on module / programme versions; subject and cost-centre splits as child tables that must total 100%	1.1, 1.3
1.3	Module versioning with effective dates	Module → ModuleVersion (what is taught; draft / approved / retired; effective dates) → ModuleRun (a delivery in a given academic year). Same for programmes. Students pinned to the programme version they enrolled on (CMA); a version with enrolled students can't change — changes create a new version. Repeating a module is a second enrolment, not an overwrite	1.1
1.5	FTE at module level + validation	Module FTE on the module version, defaulted from credits (credits ÷ full-time credit load) and overridable. Check student FTE vs the sum of module FTE per year: off / warn / stop, with a tolerance; runs at enrolment, on lifecycle approval and in return validation	1.3, 1.4
1.7	HESA Engagement entity	The Engagement record in the return, from registration and dated history. Plus engagement evidence: supervision meetings and engagement events (both exist) and new attendance / VLE data via the integration API. Confirm with Alistair which he means (probably both)	1.4, 1.12
1.10	XML output (+ CSV / Excel)	XML generated from the entity tree and validated against the HESA collection's XSD. CSV per entity and an Excel workbook (a sheet per entity) for review. Excel can be added now to the current CSV	Multi-entity return (1.7 / 1.12)
1.12	Snapshot as at a date	Returns built for a reporting window and an as-at date: values valid on those dates, and changes recorded after the freeze excluded (via recorded_at) — Alistair's 1 August example. The spec model moves from one flat student record to linked entities	1.4
1.20	Direct submission to HESA	Confirm with HESA Liaison whether the Data Platform accepts API submissions. If yes, a submission adapter behind a "Publish" button; if not, download → upload stays the route	1.10
1.23	Oracle Cloud hosting	OCI deployment guide: compute VMs, managed or self-hosted PostgreSQL, backups and DR aligned with GrantsNow; integration with pre / post-awards through the integration API	1.22

3. Workstreams

WS1 — Curriculum catalogue & versioning

Report items: 1.1 catalogue, 1.3 versioning, 3.1, 3.2 (CMA), terminology	Client tier: 2 (structural)	Size: L
---	------------------------------------	----------------

Current state

A module belongs to one “home” programme (`taught_module.programme_id`, required, cascade delete) and can be offered to others as an elective through `module_offering`. There is no version, run or effective date — only a free-text term. Module enrolments carry an `academic_year` string.

Approach: separate identity, version and yearly run

Module	stable identity: code (e.g. RM701 "Research Methods"), owning department
ModuleVersion	v1.0, v1.1 ... what is taught: title, credits, FTE, level, assessments, HESA fields; status draft -> approved -> retired; <code>valid_from</code> / <code>valid_to</code>
ModuleRun	a delivery of a version in an academic year: 2026/27, start/end dates, term
Programme	stable identity: code (e.g. MSC-ONC)
ProgrammeVersion	structure: which ModuleVersions, core/optional, credit rules, HESA course fields; status draft -> approved -> retired; <code>valid_from</code> / <code>valid_to</code>
Cohort	students who enrolled on this ProgrammeVersion (the CMA promise)
ModuleEnrolment	student -> ModuleRun (so a repeat in 26/27 is a second enrolment, with attempt number, not an overwrite)

- **Catalogue:** modules become first-class. A `programme_version_module` link (core / optional, credits) replaces the home-programme FK and the offering table. “Research Methods” is one module linked into many programmes.
- **CMA rule:** a student is pinned to the ProgrammeVersion they enrolled on. An approved version with enrolled students is immutable; changing it means creating a new version for the next cohort. The UI shows which cohorts run on which version.
- **Repeats and interruptions:** a student who started a module in 25/26, interrupted and returned in 26/27 has two enrolments on two runs, keeping the first attempt's history.
- **Terminology:** neutral names in the data model; per-institution labels (programme / course, module / unit) as settings in the existing registry, applied by a frontend label hook. Covers the England / Scotland difference.

Migration

Each existing module becomes Module + ModuleVersion 1.0 + a run for each academic year it has enrolments in; each programme becomes ProgrammeVersion 1.0 linking its home modules (core) and offerings (optional); every existing student is pinned to version 1.0. The MSc Oncology seed data is migrated the same way.

Deliverables: catalogue screens (module library, programme builder), version lifecycle (draft / approve / retire, clone to new version), cohort pinning, migrated data, label settings.

WS2 — Module & course statutory data, credits and FTE

Report items: 1.2, 1.5, 1.6, 3.3	Client tier: 1	Size: M
---	-----------------------	----------------

Current state

Credits exist on modules and a credit total on programmes. There are no HESA fields on programmes or modules, no module FTE, and no check between student FTE and modules. Student FTE lives on lifecycle events and feeds STULOAD.

What HESA holds at course and module level

HESA's Module entity includes MODID, CRDTPTS (credit value) and FTE (proportion of FTE). HESA notes the sum of a student's module FTE usually equals their instance FTE (STULOAD); where it differs (e.g. suspensions, research students with modules spanning years), the student's FTE is apportioned across cost centres and subjects in proportion to the modules. Course-level entities (Course, CourseReference, CourseRole) carry course identifiers, subjects and cost centres.

Approach

- 1 Spec-driven statutory attributes.** Course and module HESA fields are defined by the active spec pack (as student fields are today), not hard-coded. Frequently used ones (credits, FTE, level, mode) are real columns on ModuleVersion / ProgrammeVersion; subject and cost-centre splits are child tables with percentages that must total 100%.
- 2 Module FTE** on ModuleVersion, defaulted from credits (credits ÷ a configurable full-time credit load, e.g. 180 for PGT) and overridable.
- 3 FTE check** per student per academic year: student FTE vs the sum of module FTE on their runs that year. The client's rule is "student FTE must not exceed the sum of module FTEs"; HESA's note says they usually match. Implement a configurable check (setting `statutory.fte_check`: off / warn / stop, with a tolerance) that runs at module enrolment, on lifecycle approval and in the return's validation report.
- 4 Research students** (no modules): the check is skipped or uses a notional research module, per the answer to open question Q2.

Depends on WS1 (fields live on versions) and WS3 (FTE for the period, not the latest).

WS3 — Effective dating everywhere

Report items: 1.4, 1.13, 3.4, 4.3, 4.5	Client tier: 1–2	Size: L
---	-------------------------	----------------

The detailed design is in docs/approach/effective-dating/Effective_Dating_Approach.pdf: one history pattern (valid_from / valid_to / recorded_at), one write path, cached current values, a return that reads periods. This plan adds what the meeting raised:

- **Scope widened, as the client asked ("like an HR system"):** student status, programme, intensity, module enrolments, supervisors (already dated), funding (already dated), **Unit of Assessment** and catalogue versions (WS1).
- **Unit of Assessment is new.** Nothing in the code models UOA today. Add a UOA reference list (configurable) and an effective-dated assignment to staff / supervisors (and, if needed, to students or research projects — open question Q4). Alistair's example of a supervisor's UOA changing late is exactly this.
- **Alistair's status example** (Live in Aug, interrupted in March, back in June, interrupted again in Sept; snapshot 1 Aug) becomes the golden test case for the return.

Quick win for item 1.13 (programme change not in the return)

The return already produces one record per programme period when it's built for an academic year (programme_periods_for). The demo probably generated it without a year, or the profile didn't map the per-period programme fields. Verify and fix this first — it may close a client-reported gap in days, before the full WS3 work.

WS4 — HESA entity coverage & snapshot reporting

Report items: 1.7, 1.12, 4.4	Client tier: 1	Size: L
------------------------------	----------------	---------

Current state

One built-in pack (HESA_STUDENT 2026/27, 24 starter fields) and a flat, student-level record. Spec versions store a flat field list with no entity. The return reads current values; the academic year is only used to split programme periods.

Approach: a multi-entity return

- **Entities in the spec model.** Spec fields are grouped by entity with parent/child keys (e.g. Student → Engagement → StudentCourseSession → SessionStatus / ModuleInstance / ..., plus provider-level Course and Module). The mapping screen works per entity; the existing student mappings become the Student entity.
- **Record builder becomes a tree builder** that emits one record per entity occurrence, reading effective-dated history (WS3). The flat record stays for existing profiles.
- **Snapshot “as at”.** Every return is built for a reporting window and an as-at date. Two guarantees: values are those *valid* on the dates in scope, and changes *recorded* after the freeze point don't bleed in (using recorded_at). This is what Alistair's 1 August example needs.

Engagement entity

Two related things were discussed and should be kept apart:

	HESA Engagement entity	Engagement monitoring evidence
What	The student's engagement with the provider as a return record. HESA treats a student as having started (ENGSTARTDATE) once they attended a class, accessed online materials or attended induction.	Proof the student is actually studying: attendance, VLE activity, supervision meetings.
Source	Registration data and effective-dated history (WS3)	Existing: supervision meetings, engagement events (meetings, milestones). New: attendance records and VLE activity, imported through the integration API (WS6)
Output	Engagement records in the return	Engagement dashboard, non-engagement alerts, evidence for the start date

Confirm with Alistair which of these “Engagement” means for them (Q1) — likely both.

WS5 — Return output, amendments & submission

Report items: 1.10, 1.11, 1.20, 3.6	Client tier: 1 (XML) / 3 (edit, API)	Size: M
--	---	----------------

Current state

CSV only (no XML, no Excel). Each generated return is stored as an export job and can be re-downloaded, but it doesn't record the validation report, profile version or as-at date, and can't be amended.

Approach

- 1 XML** generated from the entity tree (WS4) and validated against the collection's XSD from the HESA coding manual before download. **CSV** per entity and an **Excel** workbook (a sheet per entity) for human review — same data, three renderings.
- 2 Submission versions.** Each generated return is an immutable version: XML, CSV, validation report, as-at date, profile and spec version, who generated it. Versions of the same return can be compared field by field.
- 3 Amendments (the late-correction workflow).** Rather than hand-editing raw XML, the user starts an amendment from a previous version and overrides specific fields on specific records (e.g. one supervisor's UOA), each with a reason. The new XML is rebuilt from the **previous snapshot plus those overrides**, so out-of-scope changes made since don't creep in — exactly Alistair's concern. A diff against the previous version is shown before download.
- 4 Raw XML escape hatch.** For rare cases, download, edit and re-upload the XML; on upload it's re-validated against the XSD and stored as a new, audited version.
- 5 Direct submission (Tier 3).** HESA collections are submitted through the HESA Data Platform. Whether it offers an API we can call must be confirmed with HESA Liaison (Q6). If it does, a submission adapter sits in the integration layer behind a “Publish” button; until then, users download and upload.

WS6 — Integration API & data publication

Report items: 1.9 (a) warehouse, (b) object API, 3.5	Client tier: 1 (“essential”)	Size: L
---	-------------------------------------	----------------

Current state

An integration hub with Finance / HR / Research adapters, an outbox with dispatch and dead-letter replay, signed inbound webhooks and OpenAPI docs. No API keys or service accounts for third parties, and no scheduled publication.

(b) Object-based API

- Versioned external API (/api/ext/v1/{object}) for Student, Person, Programme, Module, Enrolment, Supervision, Funding and more, defined as published objects.
- Service accounts with client-credential tokens, per-object read / write scopes, rate limits, pagination, idempotency keys, audit of every call.
- Writes go through the same services as the UI, so validation, approvals and effective dating apply — no back door.
- Change notifications through the existing outbox / webhooks.

(a) Scheduled data publication

- “Publications” configured in the admin console: object, fields, frequency, destination (SFTP, object storage, database / warehouse), format (CSV, JSON, Parquet).

- Incremental extracts using updated_at / the change feed; full extract on demand. Run by the existing worker scheduler with a publication log and retry.
- Read from a read-only session (or a replica on larger installs) so the app isn't slowed — the reason direct DB access was declined.

WS7 — Custom attribute governance

Report items: 1.18, 4.1 (all five asks)	Client tier: 2	Size: M
---	----------------	---------

The approval gate is designed in docs/approach/custom-attribute-problem/Custom_Attribute_Approval_Gate.pdf. Mapping the meeting's five asks onto it:

Client ask	Response
1. Architecture / schema document	WS8 — includes how custom attributes are stored, loaded and limited
2. Maker–checker approval	Gate layer 3: request → second-person approval; requester can't approve
3a. Warn about slowdown	Gate shows the attribute's cost: how many screens / returns / publications would load it, and the tenant's active-attribute count against a soft cap
3b. Warn if derivable from existing data	Gate layer 1 (duplicate / core-field check) extended to “can be derived from X” suggestions
3c. “Spider web” of usage	NEW: a usage graph per attribute — mappings, screens, reports, publications, API objects — shown before any change or retirement
4. Core vs organisation-specific	Category on each attribute + a promotion path: a widely used attribute is promoted to a core column in a normal release
5. Fast joins	Load only the attributes a screen/return actually uses (never all); one keyed query per request; per-entity materialised projection for reporting

Fix now
delete_field deletes an attribute without checking whether a statutory mapping uses it. Add the same usage guard as required statutory fields (409 with the list of users).

WS8 — Schema, HESA coverage & architecture documentation

Report items: 1.8, 4.1 ask 1, 4.2	Client tier: 2	Size: S
--	-----------------------	----------------

- **HESA coverage document, generated, not hand-written.** The record-schema catalog, spec packs and mappings already hold the facts. Generate a per-spec-version document (PDF and an in-app page): each HESA field → internal source → transform / default → covered / not covered / custom attribute. It stays current as mappings change.
- **Architecture document:** modular-monolith layout, data model, tenancy (RLS), effective dating, custom-attribute storage and limits, integration layer, deployment.
- **Engineering quality evidence** for Ashley's AI-code concern: module boundaries, test suite (~560 automated tests today), type checks, migrations, code review and QA gates, backward-compatibility policy. Evidence, not assurances.

WS9 — Platform: installs, feature flags, hosting, regulatory updates

Report items: 1.21, 1.22, 1.23, 3.8–3.10	Client tier: 4	Size: M
---	-----------------------	----------------

Topic	Current state	Approach
Federated installs, one code base	Shared-DB multi-tenancy with RLS	One tenant per customer VM — no code change. RLS stays as defence in depth; shared hosting remains possible for smaller customers.
Features off by default	Settings registry; <code>composer.enabled</code> is the precedent	A feature registry: every new capability ships with a flag, default off; an admin-console page to enable; enforced in the API (dependency) and UI (route guard / <code>useFeature</code>).
Oracle Cloud (with GrantsNow)	No OCI material; docs assume generic VMs + PostgreSQL	OCI deployment guide: compute VM(s), OCI Database with PostgreSQL or self-managed PostgreSQL, backups and DR aligned with GrantsNow; integration with pre/post-awards through WS6.
Regulatory updates under subscription	Spec packs are global tables	Release each HESA cycle's spec pack centrally, versioned; institutions adopt it through the existing advisory / spec-version flow.

WS10 — Carry-overs & next demo

Report items: 1.14–1.17, 1.19, 1.24	Client tier: 3	Size: S
-------------------------------------	----------------	---------

- **Milestones (1.19):** built — templates per programme, per-student panel. Demo it next time; no build needed.
- **Cohort import, single enrolment, lifecycle, advisory (1.14–1.17):** built. Fields sourced from other systems at enrolment arrive via WS6.
- **Next demo — finance payments flow (1.24):** build on the existing Finance adapter: student → pre-award / post-award (GrantsNow) → payments, through WS6. Needs GrantsNow endpoints and a test environment.

4. Dependencies and sequencing

FOUNDATIONS (no dependencies)		
WS3	Effective dating	
WS1	Catalogue & versions	
WS9	Feature flags	every new feature ships behind one
BUILT ON THEM		
WS2	Module statutory + FTE	needs WS1, WS3
WS4	Entities + snapshot	needs WS2, WS3
WS5	XML, versions, amendments	needs WS4
INDEPENDENT (parallel)		
WS6	Integration API + publication	
WS7	Custom attribute governance	(delete guard now)
WS8	Documentation	(early; regenerated as others land)

- **Why foundations first:** XML, snapshot reporting and module FTE all read versioned, effective-dated data. Building them on today's “current value” model would mean rebuilding them.
- **How we still show Tier 1 progress:** thin vertical slices — e.g. status history → SessionStatus in the return → XML for that entity — so each demo shows an end-to-end Tier 1 result.

5. Phased roadmap

Phase	Scope	Client-visible outcome
0 — Quick wins	Verify / fix programme periods in the return (1.13); custom-attribute delete guard; feature registry; Excel output; generated HESA coverage document; architecture + QA document; demo milestones	Two reported gaps closed; the documents they asked for
1 — Foundations	Effective-dating core (history service, status history, leak fixes, backfill); module catalogue + versions + runs; cohort pinning; terminology labels	Status history visible; catalogue and version screens; CMA-safe cohorts
2 — Statutory depth	Course / module HESA fields; module FTE + configurable check; multi-entity spec model; Engagement; snapshot as-at; UOA	Alistair's 1 August example returns correctly; FTE check warns / stops
3 — Output	XML + XSD validation; submission versions; amendment workflow; raw-XML escape hatch	Submission-ready XML; late corrections without scope creep
4 — Integration	External object API + service accounts; scheduled publications (can start alongside phase 1)	Essential integration asks met
5 — Governance & platform	Custom-attribute gate (approval, usage graph, categorisation); OCI deployment; HESA direct submission if an API exists; finance payments demo	Governed flexibility; hosting aligned with GrantsNow

6. Questions to confirm with the client

Best raised before Ashley and Alistair consolidate feedback with Stuart.

#	Question	Why it matters
Q1	By “Engagement” do you mean HESA's Engagement entity, attendance / VLE monitoring, or both? Which attendance / VLE systems would feed it?	WS4 scope
Q2	Exact FTE rule: student FTE $\leq$ sum of module FTE, or equal within a tolerance? How are research students (no modules) treated?	WS2 validation
Q3	Which changes to a module or programme force a new version (credits? assessment? title?) and who approves a version?	WS1 versioning rules
Q4	UOA: REF Unit of Assessment on staff / supervisors only, or also on students or projects? Is it reported to HESA?	WS3 scope
Q5	Which HESA collections are in scope (Data Futures Student; others)? Scottish institutions too?	WS4 entities, terminology
Q6	Does your HESA Data Platform access allow API submission, or is upload the only route?	WS5 direct submission
Q7	Warehouse target (Oracle ADW, Azure, Snowflake, SQL Server...) and delivery method; which objects first?	WS6 publication
Q8	For late corrections: are field-level amendments enough, or do you need to edit raw XML?	WS5 amendment design

7. Risks

Risk	Mitigation
Foundations (WS1, WS3) delay visible Tier 1 progress	Thin vertical slices; quick wins in phase 0
Catalogue / versioning migration breaks existing taught data	Additive migration, v1.0 of everything, golden tests on the MSc Oncology data
HESA spec detail larger than expected (many entities)	Spec-driven model; entities added per collection, not hard-coded
Custom attributes grow unchecked before governance lands	Delete guard now; soft cap + approval early in phase 5 or sooner
Direct HESA submission not possible	Download / upload remains the supported route
Scope creep from the Stuart feedback round	Re-plan against this matrix once their consolidated feedback arrives

8. Sources

Meeting: SMS_Demo2_Report.docx (29 September 2026). Code checked on 30 September 2026. Related designs: Effective_Dating_Approach.pdf, Custom_Attribute_Approval_Gate.pdf, Multi_Tenancy_Approach.pdf.

- [HESA — Module entity \(MODID, CRDTPTS, FTE\)](#)
- [Oracle — Data Futures Return, Module entity](#)
- [Oracle — Data Futures Return, Course entity](#)
- [Oracle — Data Futures Return, Engagement entity](#)
- [HESA coding manual — ENGSTARTDATE](#)
- [HESA — Data Futures support guides \(Data Platform, XML tools\)](#)
- [Oracle — Generating a HESA return and creating a return file](#)